ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



TRANSVIDCLOUD - NỀN TẢNG XỬ LÝ VÀ PHÂN PHỐI VIDEO TRÊN AWS

BÁO CÁO TIẾN ĐỘ LẦN HAI - NHÓM 12

MÔN HỌC: ĐIỆN TOÁN ĐÁM MÂY

Giảng viên:	TS. Hoàng Xuân Tùng
Mã môn học:	INT3319 1
Thành viên:	Đào Lê Long Vũ - 23021750 Bùi Hải Phương - 23021666 Nguyễn Hữu Lưu - 23021617 Hồ Văn Tiến Nguyên - 23021649

HÀ NỘI - 2026

Mục lục

Mục lục . . . . . **ii**

Danh sách hình ảnh . . . . . **iv**

Danh sách bảng . . . . . **v**

1 Giới thiệu **1**

1.1. Giới thiệu chung về dự án TransVidCloud 1

1.2. Đối tượng người dùng và các bên liên quan 1

1.2.1. Người dùng ứng dụng SaaS trực tiếp 2

1.2.2. Nhà phát triển và đội ngũ kỹ thuật của ứng dụng bên ngoài . . 3

1.2.3. Khách hàng tham chiếu: nền tảng giáo dục trực tuyến 4

2 Giải pháp thực hiện (chính thức) **6**

2.1. Kiến trúc tính năng của hệ thống 6

2.1.1. Phân hệ quản lý người dùng và xác thực 6

2.1.2. Phân hệ quản lý dự án và tài nguyên video 8

2.1.3. Phân hệ tiếp nhận và lưu trữ video 8

2.1.4. Phân hệ điều phối tác vụ xử lý video 8

2.1.5. Phân hệ xử lý và chuyển mã video 9

2.1.6. Phân hệ quản lý trạng thái và theo dõi tác vụ 9

2.1.7. Phân hệ phân phối và khai thác video đầu ra 10

2.1.8. Luồng xử lý chức năng tổng thể 10

2.2. Kiến trúc triển khai của hệ thống 11

2.2.1. Tổng hợp thay đổi so với Báo cáo 1 11

2.2.2. Tổng quan mô hình triển khai 12

2.2.3. Sơ đồ kiến trúc triển khai 12

2.2.4. Chi tiết các lớp triển khai 12

2.2.5. Giao diện tương tác giữa các thành phần 14

2.2.6. Luồng xử lý video chính 16
2.2.7. Vận hành, giám sát và CI/CD 17
2.2.8. Chịu lỗi, nhất quán và co giãn 17
2.2.9. Đánh đổi và phương án dự phòng 19

3 Kế hoạch triển khai và ước tính nỗ lực 20
3.1. Cơ sở kế hoạch 20
3.2. Kế hoạch triển khai 21

Danh sách hình ảnh

1.1 Đối tượng người dùng và các bên liên quan của TransVidCloud 2
1.2 Biểu đồ ca sử dụng của người dùng SaaS trực tiếp 3
1.3 Biểu đồ ca sử dụng của nhà phát triển và ứng dụng bên ngoài 4

2.1 Kiến trúc tính năng dự kiến của TransVidCloud 7
2.2 Kiến trúc triển khai thời gian chạy của TransVidCloud trên AWS. Nét liền: giao tiếp đồng bộ; nét đứt: bất đồng bộ hoặc điều khiển; [n]: số giao diện trong Bảng 2.2. 13
2.3 Luồng CI/CD, giám sát và điều khiển co giãn Worker. Nét liền: luồng chính; nét đứt: điều khiển hoặc quan sát; [n]: số giao diện trong Bảng 2.2. 18

3.1 Master Timeline triển khai dự án TransVidCloud trong bốn sprint . . . 22

Danh sách bảng

2.1 Tổng hợp thay đổi so với Báo cáo 1 11
2.2 Các giao diện tương tác chính của TransVidCloud 14

3.1 Ước tính năng lực triển khai của nhóm TransVidCloud 20
3.2 Vai trò và trách nhiệm của các thành viên 21
3.3 Kế hoạch triển khai TransVidCloud 21

Chương 1

Giới thiệu

Ghi chú: Không thay đổi nội dung Chương 1 so với Báo cáo tiến độ 1.

1.1 Giới thiệu chung về dự án TransVidCloud

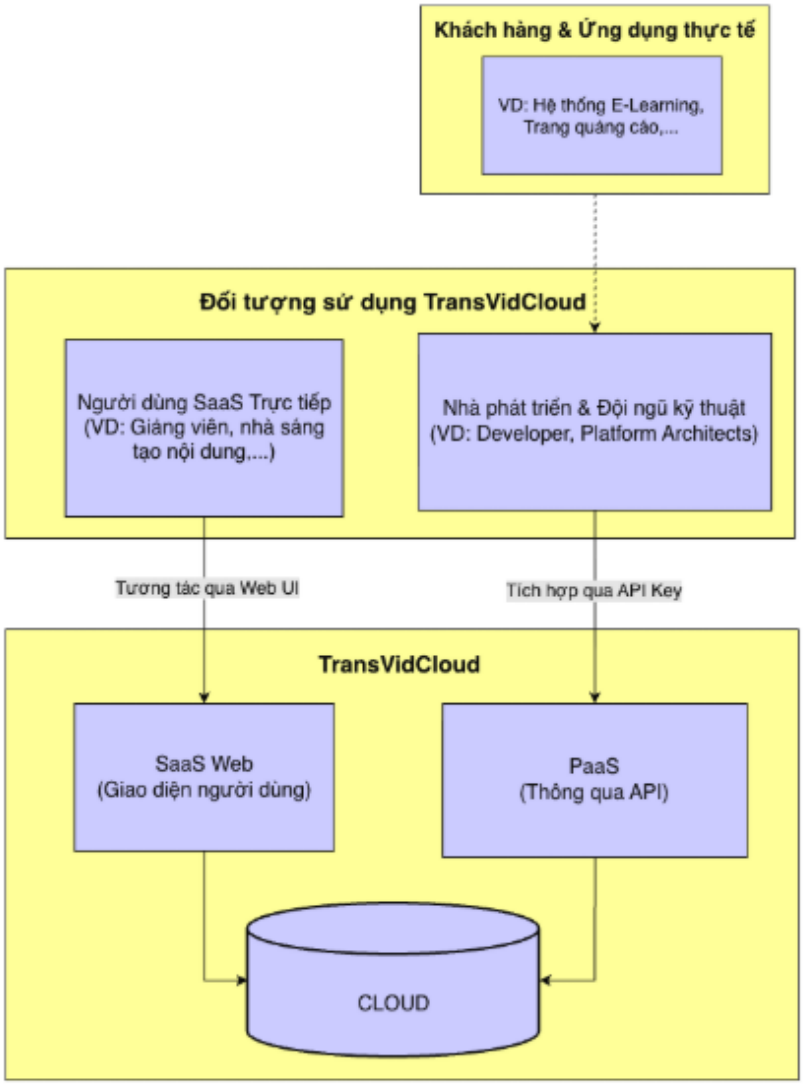
TransVidCloud là nền tảng xử lý và phân phối video trên nền tảng đám mây, được xây dựng nhằm giải quyết nhu cầu xử lý video có yêu cầu cao về tài nguyên tính toán và lưu trữ. Dự án kết hợp mô hình SaaS dành cho người dùng cuối (các đối tượng cụ thể được trình bày tại Mục 1.2) với Developer API dành cho các hệ thống phần mềm bên thứ ba. Hai phương thức sử dụng cùng vận hành trên một lõi xử lý video phân tán.

TransVidCloud tập trung giải quyết đặc thù của các tác vụ xử lý video, vốn tiêu tốn nhiều CPU và có khối lượng công việc biến động mạnh, có thể tăng đột biến khi nhiều video được tải lên cùng lúc. Hệ thống hỗ trợ tiếp nhận và lưu trữ an toàn video gốc, xử lý bất đồng bộ để tránh gây quá tải cho máy chủ ứng dụng, chuyển đổi video thành nhiều phiên bản có độ phân giải và tốc độ bit khác nhau, đóng gói phục vụ phát trực tuyến như HLS, tạo ảnh thu nhỏ và lưu trữ siêu dữ liệu. Bên cạnh đó, hệ thống cung cấp cơ chế xử lý lỗi và thử lại, kết hợp khả năng tự động co giãn theo khối lượng công việc nhằm tối ưu chi phí tài nguyên, thay vì phải duy trì một cụm máy xử lý video lớn hoạt động liên tục ngay cả khi không có yêu cầu.

1.2 Đối tượng người dùng và các bên liên quan

TransVidCloud hướng tới hai nhóm người dùng chính, với nhu cầu và đặc thù sử dụng khác nhau. Các nhóm đối tượng và mối quan hệ với nền tảng được minh họa trong

Hình 1.1.



Hình 1.1: Đối tượng người dùng và các bên liên quan của TransVidCloud

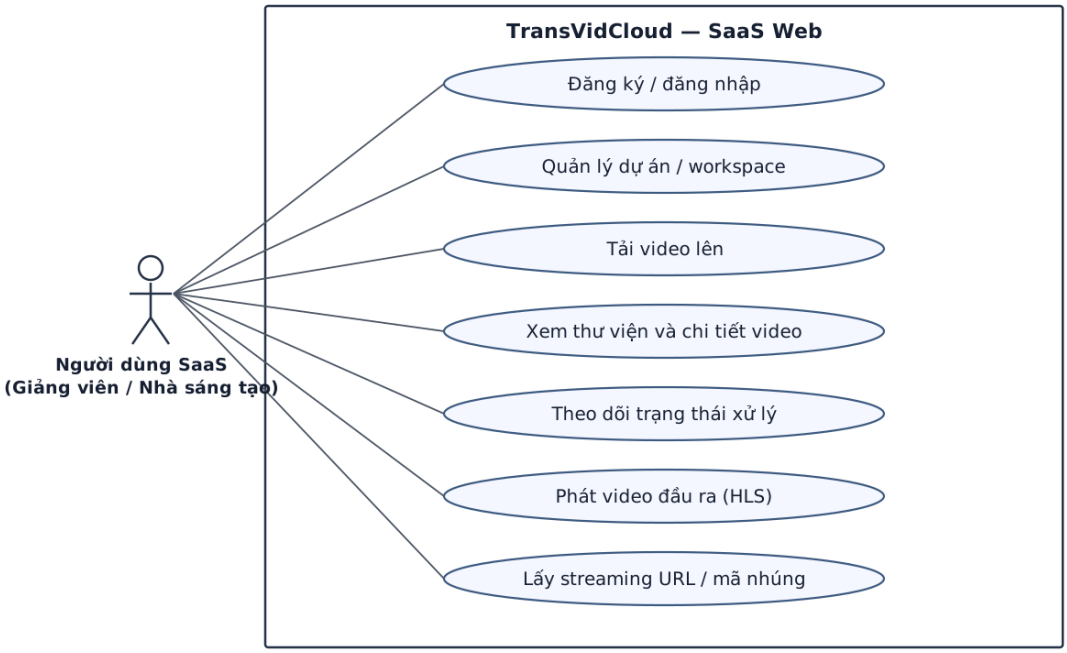
1.2.1 Người dùng ứng dụng SaaS trực tiếp

Nhóm người dùng mục tiêu của TransVidCloud gồm các cá nhân và tổ chức có nhu cầu lưu trữ, xử lý và phân phối video trực tuyến, chẳng hạn như nhà sáng tạo nội dung, giảng viên, giáo viên trực tuyến và bộ phận truyền thông số tại doanh nghiệp hoặc tổ chức. Đây là những đối tượng không nhất thiết có kiến thức chuyên sâu về công nghệ đám mây hoặc kỹ thuật xử lý video. Vì vậy, họ cần một nền tảng có giao diện trực quan, thao tác đơn giản và hỗ trợ toàn bộ quy trình từ tải video lên, quản lý nội dung đến theo dõi quá trình xử lý.

TransVidCloud giúp người dùng quản lý và phân phối video trên một nền tảng

duy nhất: tổ chức video theo dự án hoặc mục đích sử dụng, tải lên và lưu trữ video dung lượng lớn, theo dõi trạng thái xử lý, xem trước và khai thác video đầu ra. Người dùng có thể sử dụng các phiên bản video với chất lượng khác nhau, lấy đường dẫn phát trực tuyến hoặc mã nhúng để tích hợp vào website, bài viết và các nền tảng khác. Qua đó, nền tảng giảm sự phụ thuộc vào máy tính cá nhân trong quá trình xử lý video, đồng thời đơn giản hóa việc cung cấp nội dung phù hợp với nhiều thiết bị và nhu cầu sử dụng.

Các chức năng chính dành cho nhóm người dùng này được minh họa trong Hình 1.2.



Hình 1.2: Biểu đồ ca sử dụng của người dùng SaaS trực tiếp

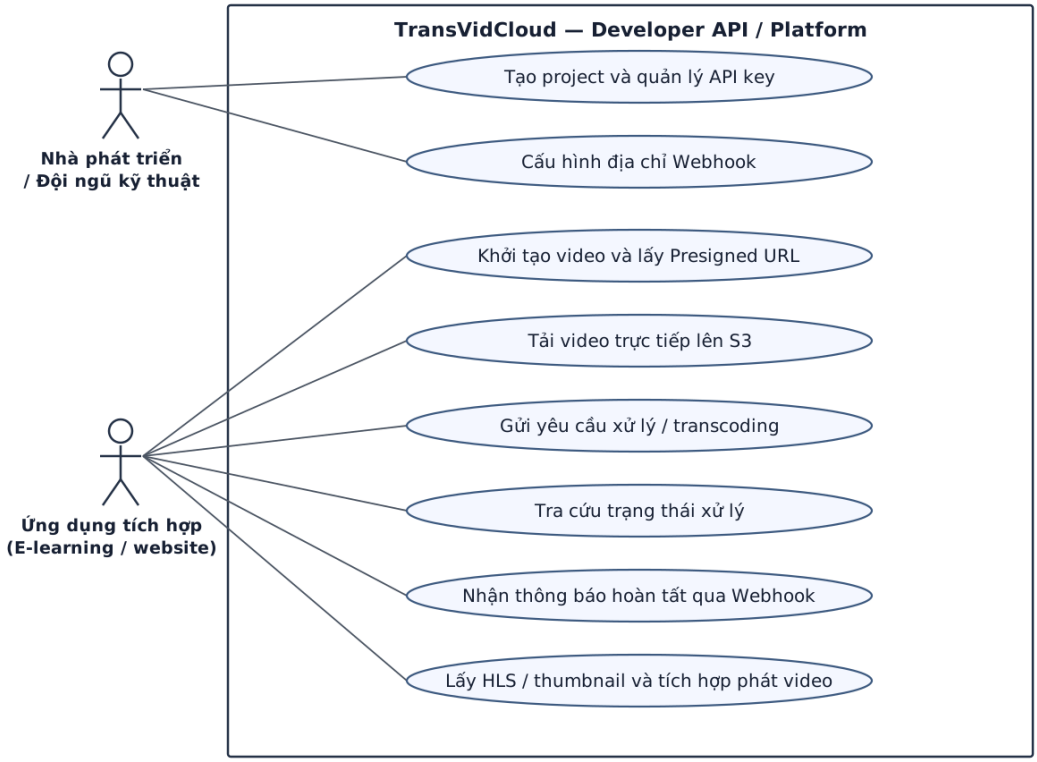
1.2.2 Nhà phát triển và đội ngũ kỹ thuật của ứng dụng bên ngoài

Nhóm đối tượng này gồm các kỹ sư phần mềm, kiến trúc sư giải pháp và đội ngũ kỹ thuật vận hành hệ thống web hoặc ứng dụng di động có nhu cầu xử lý, phân phối video. Các hệ thống này có thể thuộc nhiều lĩnh vực, từ thương mại điện tử và mạng xã hội đến cổng thông tin và doanh nghiệp truyền thông. Thay vì tự xây dựng và duy trì hạ tầng xử lý video, họ cần một nền tảng cung cấp API và dịch vụ xử lý video sẵn có, dễ tích hợp vào hệ thống hiện tại và hỗ trợ tự động hóa quy trình từ tải lên, xử lý đến phân phối video.

TransVidCloud cung cấp cơ chế xác thực, tải video trực tiếp lên lưu trữ đám mây,

khởi tạo và theo dõi tác vụ xử lý, truy xuất tài sản video đầu ra và nhận thông báo qua Webhook khi quá trình xử lý hoàn tất. Video sau xử lý có thể được tích hợp trực tiếp vào ứng dụng thông qua đường dẫn phát trực tuyến hoặc đường dẫn nhúng. Nhờ đó, nền tảng giúp doanh nghiệp rút ngắn thời gian phát triển, giảm công sức vận hành hạ tầng xử lý video riêng, tối ưu chi phí tài nguyên và hạn chế nguy cơ quá tải hệ thống ứng dụng khi lượng video tải lên tăng đột biến.

Các chức năng tích hợp dành cho nhà phát triển và ứng dụng bên ngoài được minh họa trong Hình 1.3.



Hình 1.3: Biểu đồ ca sử dụng của nhà phát triển và ứng dụng bên ngoài

1.2.3 Khách hàng tham chiếu: nền tảng giáo dục trực tuyến

Để minh họa khả năng ứng dụng thực tế của TransVidCloud, một hệ thống đào tạo trực tuyến được sử dụng làm tình huống tham chiếu:

1. Giảng viên sử dụng hệ thống E-learning để đăng tải video bài giảng.

2. Thay vì để máy chủ E-learning tự giải mã và nén video — hoạt động có thể làm nghẽn các dịch vụ khác như làm bài tập và thi cử — hệ thống E-learning gửi yêu cầu xử lý đến TransVidCloud thông qua Developer API.

3. Sau khi được xử lý phân tán trên AWS, video sẵn sàng phát cho nhiều học viên trên trình duyệt và thiết bị di động. Cơ chế phát trực tuyến thích ứng băng thông (*Adaptive Bitrate Streaming*) giúp điều chỉnh chất lượng phát theo điều kiện mạng.

Chương 2

Giải pháp thực hiện (chính thức)

2.1 Kiến trúc tính năng của hệ thống

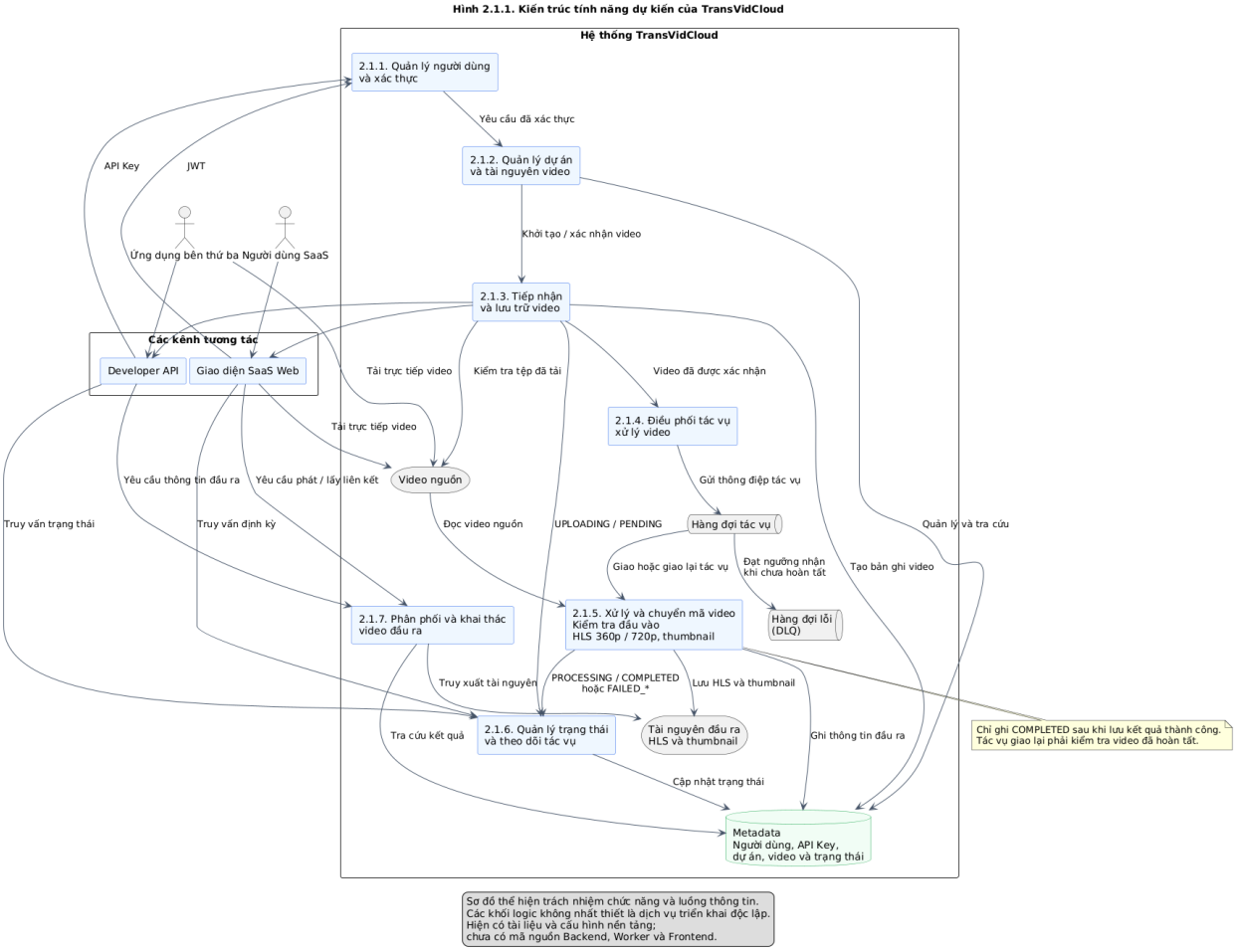
TransVidCloud tiếp tục sử dụng kiến trúc phân tách các nhóm chức năng đã trình bày trong báo cáo tiến độ lần 1. Các nghiệp vụ quản lý người dùng, quản lý dự án, tiếp nhận video và điều phối tác vụ được tách biệt với quá trình chuyển mã video tiêu tốn nhiều tài nguyên tính toán. Hệ thống hướng tới hai phương thức tương tác: giao diện Web dành cho người dùng SaaS và Developer API dành cho các ứng dụng bên thứ ba. Hai phương thức này sử dụng chung các chức năng nghiệp vụ và pipeline xử lý video.

So với Progress Report 1, cấu trúc các phân hệ không thay đổi. Thiết kế hiện tại đã được cụ thể hóa bằng khung API, cấu trúc dữ liệu, thông điệp tác vụ, quy tắc chuyển trạng thái và giới hạn video đầu vào. Tuy nhiên, các chức năng ứng dụng chưa được triển khai thành luồng xử lý hoàn chỉnh.

Hình 2.1 mô tả mối quan hệ giữa bảy phân hệ chức năng. Hai kênh SaaS Web và Developer API sử dụng chung các chức năng nghiệp vụ. Quá trình xử lý video được tách khỏi yêu cầu trực tiếp bằng hàng đợi, trong khi metadata, video nguồn và tài nguyên đầu ra được quản lý theo từng mục đích sử dụng.

2.1.1 Phân hệ quản lý người dùng và xác thực

Phân hệ này chịu trách nhiệm quản lý danh tính và kiểm soát quyền truy cập vào tài nguyên hệ thống. Người dùng SaaS dự kiến đăng nhập và xác thực thông qua JWT, trong khi ứng dụng bên thứ ba sử dụng API Key để truy cập Developer API. Các



Hình 2.1: Kiến trúc tính năng dự kiến của TransVidCloud

thao tác liên quan đến dự án và video phải được kiểm tra quyền trước khi thực hiện.

Thay đổi và bổ sung so với Progress Report 1. Thiết kế đã xác định các nhóm API đăng ký, đăng nhập và tạo API Key theo dự án. Mô hình dữ liệu đã mô tả riêng thông tin người dùng và API Key; khóa tra cứu API Key được thiết kế dưới dạng `keyHash`. Phạm vi giao diện MVP được làm rõ: có màn hình đăng nhập, còn đăng ký trực tiếp trên Web chưa thuộc phạm vi MVP.

2.1.2 Phân hệ quản lý dự án và tài nguyên video

Phân hệ này cho phép người dùng tổ chức video theo từng dự án. Mỗi video được liên kết với một dự án và có bản ghi metadata phục vụ quản lý thông tin đầu vào, theo dõi trạng thái và truy xuất tài nguyên sau xử lý.

Thay đổi và bổ sung so với Progress Report 1. Thiết kế đã xác định các thao tác tạo dự án, liệt kê dự án, liệt kê video theo dự án và xem chi tiết video. Khung API danh sách video đã đề cập cơ chế phân trang bằng cursor. Mô hình dữ liệu được cụ thể hóa bằng các bảng `projects`, `videos` và chỉ mục phục vụ truy vấn dự án theo chủ sở hữu, video theo dự án.

2.1.3 Phân hệ tiếp nhận và lưu trữ video

Phân hệ tiếp nhận video tiếp tục sử dụng Presigned URL để Client tải tệp trực tiếp vào vùng lưu trữ đầu vào. Backend có trách nhiệm khởi tạo bản ghi video, cấp đường dẫn tải lên và tiếp nhận yêu cầu xác nhận sau khi Client hoàn tất truyền tệp.

Thay đổi và bổ sung so với Progress Report 1. Quy trình xác nhận được làm rõ: Backend dự kiến kiểm tra tệp bằng thao tác `HeadObject` trước khi chuyển trạng thái từ `UPLOADING` sang `PENDING`. Thiết kế bổ sung giới hạn đầu vào gồm dung lượng tối đa 209.715.200 byte, thời lượng 300 giây và chiều cao khung hình tối đa 1080 pixel. Presigned URL dự kiến có hiệu lực 15 phút; video nguồn được cấu hình lưu giữ 7 ngày.

Việc kiểm tra được phân chia theo trách nhiệm: Frontend hỗ trợ phát hiện sớm lỗi lựa chọn tệp; Backend kiểm tra thông tin khai báo và đối tượng đã tải lên; Worker kiểm tra đặc tính thực tế của video.

2.1.4 Phân hệ điều phối tác vụ xử lý video

Phân hệ điều phối tiếp nhận video đã được xác nhận và tạo tác vụ xử lý bất đồng bộ. Tác vụ được đưa vào hàng đợi để Worker tiếp nhận, giúp tách thời gian phản hồi

của API khỏi thời gian chuyển mã video.

Thay đổi và bổ sung so với Progress Report 1. Thông điệp tác vụ đã có schema bản nháp với các trường bắt buộc `schemaVersion`, `jobId`, `videoId`, `projectId` và `inputKey`. Cơ chế tiếp nhận tác vụ được cụ thể hóa bằng hàng đợi chính và hàng đợi lỗi — Dead Letter Queue (DLQ).

Cấu hình hiện tại quy định thời gian ẩn thông điệp sau khi nhận là 300 giây, thời gian long polling là 20 giây và ngưỡng nhận thông điệp trước khi chuyển sang DLQ là 3 lần. Đây là cấu hình giao nhận tác vụ, chưa đồng nghĩa với việc logic xử lý lỗi trong Worker đã hoàn thành.

2.1.5 Phân hệ xử lý và chuyển mã video

Phân hệ này thực hiện kiểm tra video nguồn, chuyển mã sang các phiên bản 360p và 720p, đóng gói HLS và tạo thumbnail. Kết quả được lưu vào vùng đầu ra và liên kết với metadata của video tương ứng.

Thay đổi và bổ sung so với Progress Report 1. Phạm vi đầu ra được giữ nguyên, đồng thời thiết kế đã làm rõ bước kiểm tra đầu vào bằng `ffprobe` trước khi chuyển mã bằng FFmpeg. Các trường hợp lỗi được phân nhóm thành lỗi kiểm tra video, lỗi chuyển mã và lỗi lưu trữ.

Quy tắc xử lý tác vụ được giao lại cũng được cụ thể hóa: Worker phải kiểm tra trạng thái trước khi xử lý, bỏ qua video đã hoàn tất và sử dụng đường dẫn đầu ra ổn định. Những quy tắc này nhằm hạn chế việc xử lý lại và ghi đè kết quả.

2.1.6 Phân hệ quản lý trạng thái và theo dõi tác vụ

Phân hệ này quản lý vòng đời video và cung cấp trạng thái thống nhất cho giao diện Web và Developer API. Backend phụ trách giai đoạn tiếp nhận, còn Worker phụ trách giai đoạn xử lý.

Thay đổi và bổ sung so với Progress Report 1. Tài liệu đã xác định rõ luồng trạng thái:

$$\texttt{UPLOADING} \rightarrow \texttt{PENDING} \rightarrow \texttt{PROCESSING} \rightarrow \texttt{COMPLETED}.$$

Các trạng thái lỗi gồm:
`FAILED_VALIDATION`, `FAILED_TRANSCODE` và `FAILED_STORAGE`.

Thiết kế quy định `COMPLETED` là trạng thái cuối, không được ghi đè khi tác vụ được giao lại. Những lần Worker nhận tác vụ dự kiến được ghi nhận bằng `attempt-`

Count.

Đối với giao diện Web, tài liệu đề xuất truy vấn trạng thái định kỳ khoảng 3–5 giây khi còn video chờ hoặc đang xử lý. Đây là cơ chế cập nhật trạng thái; chưa có thiết kế đo phần trăm tiến độ chuyển mã thực tế.

2.1.7 Phân hệ phân phối và khai thác video đầu ra

Phân hệ này cung cấp tài nguyên video sau xử lý, gồm playlist HLS, các phân đoạn video và thumbnail. Người dùng SaaS dự kiến xem video trên Web và sao chép liên kết streaming; ứng dụng bên ngoài lấy thông tin tài nguyên thông qua Developer API.

Thay đổi và bổ sung so với Progress Report 1. Thiết kế đã xác định việc sử dụng master playlist để truy cập các phiên bản chất lượng và trình phát HLS trên Web. Môi trường local đã có cấu hình đường dẫn `/media/` trỏ tới vùng lưu trữ đầu ra. Phạm vi khai thác kết quả cũng được làm rõ: phát HLS và lấy liên kết là chức năng trọng tâm; mã nhúng và trang nhúng có thể được cắt giảm tùy tiến độ.

2.1.8 Luồng xử lý chức năng tổng thể

Luồng xử lý tổng thể tiếp tục giữ nguyên định hướng của Progress Report 1, đồng thời bổ sung rõ các bước kiểm tra và điều kiện chuyển trạng thái:

1. **Xác thực và khởi tạo:** Người dùng SaaS hoặc ứng dụng bên thứ ba xác thực, chọn dự án và gửi yêu cầu tạo video.
2. **Tiếp nhận video:** Backend kiểm tra thông tin khai báo, tạo bản ghi `UPLOADING` và cấp Presigned URL.
3. **Tải lên và xác nhận:** Client tải tệp trực tiếp vào vùng đầu vào rồi gửi yêu cầu xác nhận. Backend kiểm tra tệp trước khi chuyển video sang `PENDING`.
4. **Điều phối tác vụ:** Backend gửi thông điệp theo schema chung vào hàng đợi.
5. **Kiểm tra và xử lý:** Worker nhận tác vụ, kiểm tra trạng thái, cập nhật `PROCESSING`, xác minh video bằng `ffprobe` và thực hiện pipeline FFmpeg.
6. **Lưu kết quả:** Worker lưu HLS 360p, 720p và thumbnail, cập nhật thông tin tài nguyên rồi chuyển video sang `COMPLETED`.
7. **Truy xuất kết quả:** Client truy vấn trạng thái và lấy thông tin đầu ra để phát hoặc tích hợp video.

8. **Xử lý lỗi:** Worker ghi nhận lỗi tương ứng. Tác vụ chưa được xác nhận hoàn tất có thể được giao lại; thông điệp đạt ngưỡng nhận theo cấu hình được chuyển sang DLQ.

Thay đổi và bổ sung so với Progress Report 1. Luồng tổng thể đã được bổ sung các điểm kiểm tra trước khi đưa video vào hàng đợi, trước khi chuyển mã và trước khi ghi nhận hoàn tất. Cấu trúc thông điệp, trạng thái lỗi và quy tắc xử lý tác vụ giao lại đã có tài liệu riêng để các thành phần tuân theo khi tích hợp.

2.2 Kiến trúc triển khai của hệ thống

Kiến trúc triển khai ánh xạ các phân hệ chức năng ở mục 2.1 sang các dịch vụ cụ thể trên Amazon Web Services (AWS) và xác định cách các thành phần giao tiếp với nhau. Thiết kế dựa trên ba nguyên tắc: (1) video dung lượng lớn không đi qua máy chủ API; (2) chuyển mã video chạy nền, bất đồng bộ qua hàng đợi, người dùng không phải chờ; (3) quyền truy cập được giới hạn tối thiểu và hạ tầng được mô tả bằng mã. Các thông số như loại máy, số Worker, thời hạn URL hay ngưỡng cảnh báo là cấu hình mục tiêu của MVP và sẽ được hiệu chỉnh khi triển khai thực tế.

2.2.1 Tổng hợp thay đổi so với Báo cáo 1

Luồng xử lý cốt lõi (Client, Backend, S3, SQS/DLQ, Worker, DynamoDB) **không thay đổi**. Báo cáo 2 chỉ điều chỉnh ba quyết định triển khai, tóm tắt trong Bảng 2.1.

Bảng 2.1: Tổng hợp thay đổi so với Báo cáo 1

Nội dung	**Báo cáo 1**	**Báo cáo 2**	**Lý do và tác động**
Đường vào API	EC2 có Elastic IP, chỉ nhận HTTPS từ CloudFront	CloudFront → Cloudflare Tunnel → EC2; EC2 không mở cổng nhận từ Internet	Giảm bề mặt tấn công. Thêm một dịch vụ ngoài AWS cần kiểm thử kết nối; có phương án quay lại như Báo cáo 1.
Số Worker và loại máy	ASG 1–4 máy	1–4 máy; môi trường dev cho phép Min = 0; có thể dùng máy Spot	Tiết kiệm chi phí. Đổi lại khởi động chậm hơn khi Min = 0 và Spot có thể bị thu hồi; SQS giao lại job.
Phát video HLS	CloudFront đọc từ S3 Output	Thêm CloudFront Function bỏ tiền tố `/media`	URL phát video thống nhất. Cần kiểm thử đường dẫn trong playlist HLS để tránh lỗi 404.

2.2.2 Tổng quan mô hình triển khai

Hệ thống phục vụ hai nhóm người dùng qua một Backend dùng chung: người dùng SaaS thao tác trên giao diện React; ứng dụng bên thứ ba (ví dụ hệ thống E-learning) gọi Developer API. Amazon CloudFront là điểm vào duy nhất, phân luồng theo đường dẫn: giao diện web, API và video đầu ra. Backend Spring Boot xử lý xác thực, metadata và tạo job; cụm Worker viết bằng Go và dùng FFmpeg đảm nhiệm phần chuyển mã nặng.

Video gốc được Client tải thẳng lên Amazon S3 bằng Presigned URL do Backend cấp, nên Backend không phải truyền dữ liệu video. Khi Client xác nhận tải xong, Backend kiểm tra file, cập nhật DynamoDB và gửi thông điệp mô tả job vào Amazon SQS. Worker nhận job, xử lý, ghi HLS và thumbnail vào S3 Output rồi cập nhật trạng thái. Video đầu ra được phát qua CloudFront; các bucket S3 luôn ở chế độ private.

2.2.3 Sơ đồ kiến trúc triển khai

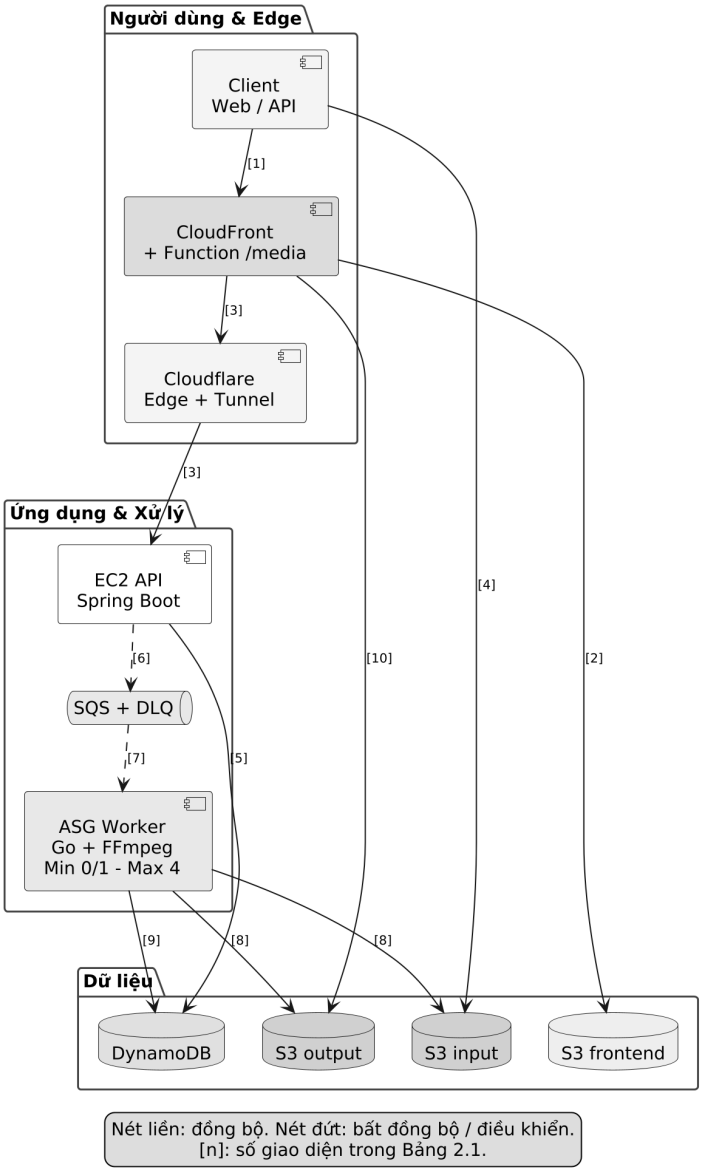
Hình 2.2 thể hiện đường chạy chính của hệ thống với các giao diện [1]–[10]. Hình 2.3 (mục 2.2.7) thể hiện phần vận hành gồm CI/CD, giám sát và co giãn với các giao diện [11]–[15]. Số [n] trong hình tương ứng với cột STT của Bảng 2.2. Các chi tiết cấu hình như IAM, Security Group, S3 Gateway Endpoint không vẽ thành nút riêng mà được mô tả trong bảng và các mục sau.

2.2.4 Chi tiết các lớp triển khai

Lớp biên và định tuyến (giao diện [1]–[3], [10]). CloudFront cung cấp một địa chỉ HTTPS với ba đường dẫn: `/` phục vụ giao diện React từ S3 Frontend; `/api/*` chuyển yêu cầu tới Backend; `/media/*` phát video từ S3 Output. Các bucket S3 để private và chỉ CloudFront được đọc thông qua Origin Access Control (OAC). Đường dẫn API không bật cache để dữ liệu trạng thái luôn mới.

Lớp API nghiệp vụ (giao diện [3]–[6]). Backend Spring Boot chạy trong Docker trên một EC2 `t3.small`. Backend quản lý xác thực, dự án và video; tạo Presigned URL; gửi job vào SQS; cung cấp API tra cứu trạng thái và tài sản đầu ra. Backend không chạy FFmpeg nên thời gian phản hồi không phụ thuộc độ dài video. Máy EC2 dùng IAM Role riêng thay vì lưu khóa AWS trong mã nguồn.

Yêu cầu `/api/*` đến Backend qua **Cloudflare Tunnel**: thành phần `cloudflared` trên EC2 chủ động mở kết nối ra Cloudflare, và Cloudflare chuyển yêu cầu từ CloudFront vào Backend qua kết nối này. Nhờ vậy EC2 không cần mở cổng nhận



Hình 2.2: Kiến trúc triển khai thời gian chạy của TransVidCloud trên AWS. Nét liền: giao tiếp đồng bộ; nét đứt: bất đồng bộ hoặc điều khiển; [n]: số giao diện trong Bảng 2.2.

kết nối từ Internet. Đây là dịch vụ ngoài AWS nên cần được kiểm thử kết nối khi triển khai.

Lớp lưu trữ và dữ liệu (giao diện [4], [5], [8], [9]). Hệ thống dùng ba bucket S3 theo mục đích: `frontend` (bản build React), `input` (video gốc, dự kiến tự xóa sau 7 ngày) và `output` (HLS và thumbnail). Metadata nằm trong bốn bảng DynamoDB On-Demand: `users`, `projects`, `apikeys`, `videos`. Bản ghi `videos` là nguồn thông tin duy nhất về trạng thái xử lý (UPLOADING, PENDING, PROCESSING, COMPLETED, FAILED). Worker truy cập S3 qua S3 Gateway Endpoint trong VPC nên không cần NAT Gateway cho lưu lượng này.

Lớp xử lý bất đồng bộ (giao diện [6]–[9]). SQS Standard là vùng đệm giữa API và Worker. Worker chạy Go và FFmpeg trong Docker, tạo HLS 360p, 720p và thumbnail. Các Worker thuộc một Auto Scaling Group (ASG) tự tăng giảm số máy theo lượng job chờ (mục 2.2.8).

Bảo mật. API dùng JWT cho người dùng SaaS và API Key cho ứng dụng tích hợp. API, Worker và pipeline triển khai dùng các IAM Role tách biệt theo nguyên tắc quyền tối thiểu. Security Group không mở cổng nhận kết nối từ Internet; quản trị máy qua SSM Session Manager, không dùng SSH. Mật khẩu và khóa bí mật lưu trong SSM Parameter Store, không đưa vào mã nguồn hay Docker image.

2.2.5 Giao diện tương tác giữa các thành phần

Bảng 2.2 mô tả 15 giao diện chính, là hợp đồng tích hợp giữa nhóm Software và nhóm Cloud. Đường dẫn API và cấu trúc dữ liệu chi tiết được tham chiếu tới tài liệu hợp đồng của nhóm Software.

Bảng 2.2: Các giao diện tương tác chính của TransVidCloud

STT	Nguồn	Đích	Giao thức và nội dung trao đổi
1	Client SaaS; ứng dụng ngoài	CloudFront	HTTPS cổng 443; một địa chỉ cho web (`/`), API (`/api/*`) và video (`/media/*`). SaaS xác thực bằng JWT, ứng dụng ngoài bằng API Key.
2	CloudFront	S3 Frontend	HTTPS GET lấy các tệp React tĩnh. Bucket private, chỉ CloudFront đọc được qua OAC.

Còn tiếp ở trang sau

Bảng 2.2 (tiếp theo)

STT	Nguồn	Đích	Giao thức và nội dung trao đổi
3	CloudFront; Cloudflare Tunnel	EC2 API	Yêu cầu `/api/*` (REST/JSON, không cache) được chuyển qua Cloudflare và đường hầm do EC2 mở sẵn, tới Backend trên cổng nội bộ 8080.
4	Backend; Client	S3 Input	Backend cấp Presigned URL có hiệu lực 15 phút. Client tải video trực tiếp lên S3 Input bằng HTTPS PUT, video không đi qua API.
5	Backend	DynamoDB; S3 Input	Khi khởi tạo, ghi bản ghi video (UPLOADING). Khi Client xác nhận tải xong, kiểm tra file đã có trên S3 rồi đổi trạng thái sang PENDING.
6	Backend	SQS	Gửi thông điệp JSON ngắn gồm phiên bản schema, `jobId`, `videoId`, `projectId` và vị trí file gốc. Worker chỉ cần hợp đồng thông điệp này, không phụ thuộc cấu trúc nội bộ của Backend.
7	SQS; Worker; DLQ	Hai chiều	Worker nhận job bằng long polling (chờ tối đa 20 giây), xử lý xong mới xác nhận (ACK) để xóa khỏi hàng đợi. Job chưa được xác nhận sau 300 giây sẽ được giao lại; job dài thì Worker gia hạn thời gian. Sau 3 lần nhận lỗi, thông điệp chuyển sang hàng đợi lỗi DLQ (giữ 14 ngày) và phát cảnh báo.
8	Worker	S3 Input; S3 Output	Đọc video gốc, ghi HLS 360p/720p và thumbnail theo quy ước đường dẫn ổn định (ví dụ `{projectId}/{videoId}/hls/master.m3u8`) qua S3 Gateway Endpoint.
9	Worker	DynamoDB	Cập nhật trạng thái PROCESSING, COMPLETED hoặc FAILED, số lần thử và danh sách tài sản đầu ra. Cập nhật có điều kiện nên video đã COMPLETED không bị ghi đè.

Còn tiếp ở trang sau

Bảng 2.2 (tiếp theo)

STT	Nguồn	Đích	Giao thức và nội dung trao đổi
10	CloudFront; CloudFront Function	S3 Output	Yêu cầu `/media/...` được bỏ tiền tố `/media` rồi đọc object qua OAC, dùng để phát playlist HLS, segment và thumbnail.
11	GitHub Actions	ECR; SSM; ASG	Kiểm thử và build image Backend/Worker, đẩy lên ECR với tag là mã commit; quyền AWS tạm thời cấp qua OIDC. Backend được cập nhật bằng SSM Run Command; với Worker, pipeline ghi tag mới vào Parameter Store rồi cho ASG thay máy dần (Instance Refresh).
12	ECR; SSM	EC2 API; Worker	Máy lấy image từ ECR; đọc cấu hình và khóa bí mật từ Parameter Store (`/tvc/{env}/...`) khi khởi động.
13	API; Worker; SQS	CloudWatch	Gửi log JSON (có `jobId`, `videoId`, giữ 7 ngày), chỉ số CPU và số thông điệp đang chờ. Khi Min = 0, tiện ích `metricpub` công bố thêm chỉ số `BacklogPerInstance`.
14	CloudWatch	ASG	Target Tracking tăng/giảm Worker trong khoảng 1–4 máy theo lượng job chờ trên mỗi máy (mục 2.2.8). Worker đang chuyển mã được bảo vệ khỏi bị thu hồi khi thu nhỏ.
15	CloudWatch Alarm; AWS Budgets	SNS	Gửi email khi DLQ có thông điệp, CPU cao hoặc job chờ quá lâu, và khi chi phí chạm ngưỡng 25/50 USD. AWS Budgets chỉ cảnh báo, không tự chặn chi tiêu.

2.2.6 Luồng xử lý video chính

1. **Khởi tạo và tải lên.** Client gọi API để tạo video; Backend kiểm tra quyền, ghi bản ghi trạng thái UPLOADING và trả Presigned URL. Client tải video thẳng lên S3 Input [4][5].

2. **Xác nhận và tạo job.** Client xác nhận đã tải xong. Backend kiểm tra file trên S3, chuyển trạng thái sang PENDING và gửi thông điệp vào SQS [5][6]. Gọi xác nhận nhiều lần không tạo thêm job.

3. **Xử lý.** Worker nhận job, chuyển trạng thái sang PROCESSING, đọc video gốc và dùng FFmpeg tạo HLS 360p, 720p cùng thumbnail [7][8]. Phạm vi MVP giới hạn video tối đa 200 MB, 300 giây và độ phân giải 1080p; giới hạn này được kiểm tra ở cả API và Worker.

4. **Công bố kết quả.** Sau khi ghi đủ tài sản vào S3 Output, Worker cập nhật trạng thái COMPLETED trong DynamoDB rồi mới xác nhận job với SQS [8][9]. Client tra cứu API để lấy trạng thái và đường dẫn phát.

5. **Phát video.** Trình phát lấy `master.m3u8` qua CloudFront tại `/media/...` [10]. Playlist này trỏ tới các bản 360p/720p để trình phát chọn chất lượng theo tốc độ mạng.

2.2.7 Vận hành, giám sát và CI/CD

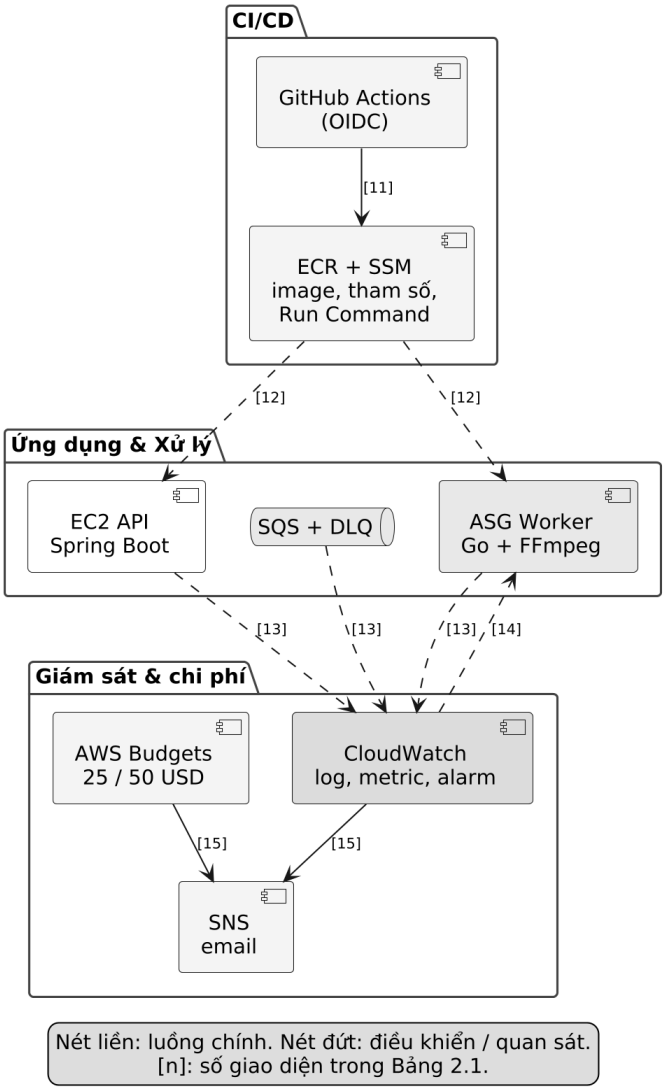
Hạ tầng được mô tả bằng Terraform, chia thành các module lưu trữ, hàng đợi, cơ sở dữ liệu, mạng, IAM, tính toán, CloudFront và giám sát. Môi trường dev và demo dùng chung mã Terraform, chỉ khác biến cấu hình.

CI/CD (giao diện [11]–[12]). GitHub Actions kiểm thử và build image Backend, Worker, rồi đẩy lên ECR với tag là mã commit. Backend được cập nhật bằng SSM Run Command, không cần SSH. Với Worker, pipeline ghi tag mới vào Parameter Store và cho ASG thay máy dần; quá trình này không được ngắt các Worker đang chuyển mã.

Giám sát và cảnh báo (giao diện [13]–[15]). CloudWatch thu log JSON, CPU và số job chờ. Cảnh báo gửi email qua SNS khi DLQ có thông điệp, CPU cao hoặc job chờ lâu. AWS Budgets cảnh báo chi phí tại 25 USD và 50 USD; đây là ngưỡng thông báo, không phải giới hạn tự động.

2.2.8 Chịu lỗi, nhất quán và co giãn

Chịu lỗi và nhất quán (giao diện [7], [9]). SQS Standard giao thông điệp *ít nhất một lần*, nên một job có thể được giao lại, chẳng hạn khi Worker bị tắt giữa chừng hoặc máy Spot bị thu hồi. Worker chỉ xác nhận sau khi đã lưu xong kết quả, vì vậy job chưa xác nhận sẽ tự xuất hiện lại sau 300 giây và một Worker khác nhận xử lý. Để job bị giao lại không gây sai lệch, Worker ghi kết quả vào cùng một vị trí trên S3 và chỉ cập nhật DynamoDB khi trạng thái còn hợp lệ; video đã COMPLETED không bị xử lý lại. Lỗi do dữ liệu đầu vào (ví dụ video không hợp lệ) được đánh dấu thất



Hình 2.3: Luồng CI/CD, giám sát và điều khiển co giãn Worker. Nét liền: luồng chính; nét đứt: điều khiển hoặc quan sát; [n]: số giao diện trong Bảng 2.2.

bại ngay và không thử lại; lỗi tạm thời được thử tối đa 3 lần rồi chuyển vào DLQ kèm cảnh báo.

Tự động co giãn (Hình 2.3, giao diện [13]–[14]). Hệ thống đo lượng job chờ trên mỗi Worker:

$$\text{BacklogPerInstance} = \frac{\text{Số thông điệp đang chờ}}{\max(1,\ \text{Số Worker đang chạy})}.$$

CloudWatch so sánh chỉ số này với giá trị mục tiêu để ASG tăng hoặc giảm số Worker trong khoảng 1–4 máy. Khi Worker xử lý job, số thông điệp chờ giảm, khép kín vòng điều khiển. Môi trường demo và đo kiểm giữ tối thiểu một Worker để có sẵn tài nguyên. Môi trường dev có thể cho Min = 0 để tiết kiệm chi phí; khi đó cần `metricpub` công bố chỉ số cả lúc không có Worker và phải kiểm thử việc tự khởi động Worker khi có job mới. Giá trị mục tiêu sẽ được hiệu chỉnh theo thời gian xử lý thực tế của một job.

2.2.9 Đánh đổi và phương án dự phòng

- **Cloudflare Tunnel:** giảm tấn công vào EC2 nhưng phụ thuộc thêm một dịch vụ ngoài AWS. Nếu không ổn định, nhóm quay lại phương án Báo cáo 1 (EC2 có Elastic IP, chỉ nhận HTTPS từ CloudFront).
- **Mạng:** EC2 đặt ở public subnet nhưng không mở cổng nhận kết nối từ Internet, thay cho private subnet kèm NAT Gateway, tiết kiệm khoảng 32 USD mỗi tháng. Đổi lại mức cô lập mạng thấp hơn nên cần siết quyền Security Group và IAM.
- **Số Worker và Spot:** Min = 0 và máy Spot giúp giảm chi phí nhưng làm tăng thời gian khởi động và có thể bị thu hồi. Vì vậy demo và benchmark dùng Min = 1; việc thu hồi máy được xử lý nhờ SQS giao lại job.
- **CloudFront:** là phương án chính; phương án dự phòng là bỏ CloudFront, cho frontend gọi trực tiếp API và S3 với cấu hình truy cập tương ứng.

Những cấu hình chưa được triển khai và kiểm thử thực tế trong báo cáo này được hiểu là thiết kế mục tiêu của MVP, không phải kết quả đã xác nhận.

Chương 3

Kế hoạch triển khai và ước tính nỗ lực

3.1 Cơ sở kế hoạch

Kế hoạch triển khai TransVidCloud gồm bốn thành viên, bốn sprint, mỗi sprint kéo dài một tuần, và một tuần dự phòng để hoàn thiện báo cáo, sửa lỗi và chuẩn bị demo. Mỗi thành viên dự kiến dành 2–3 ngày làm việc tập trung mỗi tuần trong bốn tuần triển khai. Khối lượng công việc được tính theo đơn vị person-day (pd), trong đó 1 pd tương đương một người làm việc tập trung trong một ngày.

Bảng 3.1: Ước tính năng lực triển khai của nhóm TransVidCloud

Thông số	**Giá trị**	**Ghi chú**
Số thành viên	4 người	Cùng tham gia phát triển và tích hợp.
Số sprint	4 sprint	Tương ứng bốn tuần triển khai.
Thời gian dự phòng	1 tuần	Hoàn thiện báo cáo, sửa lỗi và chuẩn bị demo.
Capacity ước tính	32–48 pd	Mỗi thành viên dành 2–3 ngày mỗi tuần trong bốn tuần.
Capacity sử dụng	40 pd	Giả định mỗi thành viên trung bình dành 2,5 ngày mỗi tuần trong bốn tuần.

Bảng 3.2: Vai trò và trách nhiệm của các thành viên

Thành viên	**Vai trò**	**Trách nhiệm**
Bùi Hải Phương	Frontend, Backend	Phát triển Backend, Developer API và giao diện SaaS.
Đào Lê Long Vũ	Frontend, Backend	Phát triển Backend, Worker và giao diện SaaS.
Nguyễn Hữu Lưu	DevOps	Thiết lập các dịch vụ Cloud.
Hồ Văn Tiến Nguyên	DevOps	Thiết lập các dịch vụ Cloud.

3.2 Kế hoạch triển khai

Workbook theo dõi của nhóm: mở Google Sheets. Bảng 3.3 tóm tắt các hạng mục, sprint dự kiến, mức ưu tiên, effort và người chịu trách nhiệm.

Bảng 3.3: Kế hoạch triển khai TransVidCloud

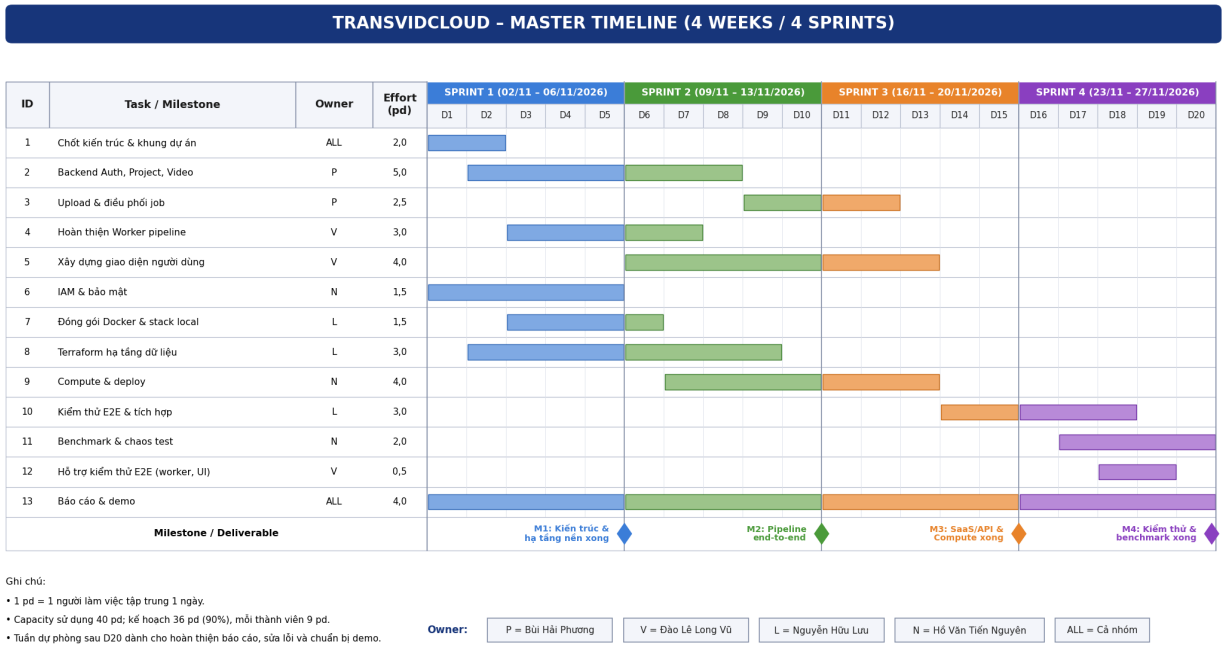
ID	**Công việc**	**Kiến trúc liên quan**	**Sprint**	**Ưu tiên**	**Effort (pd)**	**Người chịu trách nhiệm**
A01	Chốt kiến trúc và khung dự án	Toàn bộ kiến trúc	1	High	2	Cả nhóm
A02	Backend Auth, Project, Video	[1], [3], [5]	1–2	High	5	Bùi Hải Phương
A03	Upload và điều phối job	[4], [5], [6], [9]	2–3	High	2,5	Bùi Hải Phương
A04	Hoàn thiện Worker pipeline	[7], [8], [9]	1–2	High	3	Đào Lê Long Vũ
A05	Xây dựng giao diện người dùng	[1], [2], [4], [10]	2–3	High	4	Đào Lê Long Vũ
A06	IAM và bảo mật	[5], [6], [8], [9], [12]	1–2	High	1,5	Hồ Văn Tiến Nguyên
A07	Đóng gói Docker và stack local	[12]	1–2	Medium	1,5	Nguyễn Hữu Lưu
A08	Terraform hạ tầng dữ liệu	[4], [5], [8], [9]	1–2	High	3	Nguyễn Hữu Lưu

(Tiếp tục ở trang sau)

Bảng 3.3 (tiếp theo)

ID	Công việc	Kiến trúc liên quan	Sprint	Ưu tiên	Effort (pd)	Người chịu trách nhiệm
A09	Compute và deploy	[11], [12]	2–3	High	4	Hồ Văn Tiến Nguyên
A10	Kiểm thử E2E và tích hợp	[1]–[10]	3–4	High	3	Nguyễn Hữu Lưu
A11	Benchmark và chaos test	[13], [14]	4	Medium	2	Hồ Văn Tiến Nguyên
A12	Hỗ trợ kiểm thử E2E (Worker, giao diện)	[1], [7], [10]	4	Medium	0,5	Đào Lê Long Vũ
A13	Báo cáo và demo	Toàn bộ kiến trúc	1–4 + dự phòng	Medium	4	Cả nhóm
				Tổng effort kế hoạch	**36**	

Tổng effort kế hoạch là 36 pd, tương đương 90% capacity sử dụng 40 pd; mức phân bổ trung bình là 9 pd mỗi thành viên. Tuần dự phòng sau D20 dành cho việc hoàn thiện báo cáo, sửa lỗi và chuẩn bị demo.



Hình 3.1: Master Timeline triển khai dự án TransVidCloud trong bốn sprint